AWS-Arkitehtuuri Lopputyö

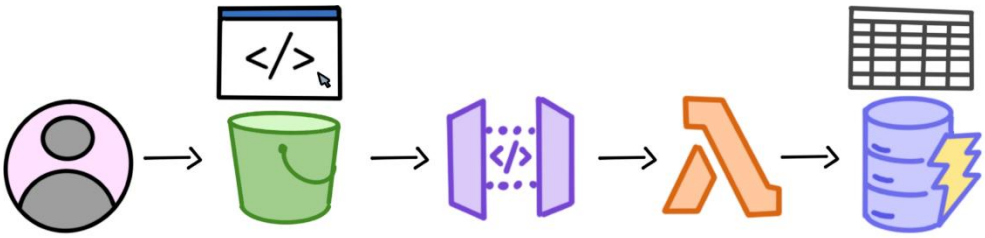
Jun Fengari

Contents

Konsepti 2
Palvelut 2
Arkkitehtuurikaavio 3
CloudFormation 4
Frontend: frontend.yaml 4
Backend: backend.yaml 4
Database: database.yaml 4
Security: waf.yaml 4
Prosessi 5
Tulokset & Reflektio 8
Lähteet & Tekoälyn Käyttö 10
Tekoälyn käyttö 10

Konsepti

Toteutin AWS-Arkkitehtuurikurssin lopputyönä tyypillisen serverless-arkkitehtuurin. Arkkitehtuurin toiminnallinen päätavoite oli yhdistää käyttäjän interaktio verkkosivulla DynamoDB- tietokantaan.



Palvelut

Arkkitehtuurissa käytetyt palvelut (5):
S3, DynamoDB, Lambda, API-Gateway, WAF

Hallintapalvelut (5):
CloudWatch, CloudFormation (& IAM, AWS Management Console, CloudShell)

Suunnittelussa mutta ei toteutuksessa käytetyt palvelut (1):
CloudFront

(*Architecture Icon Page*, ei pvm.)

Mitä AWS-palveluja käytit ja miksi?

Käytin DynamoDB tietokantaa, sillä minulla oli siitä eniten kokemusta ja valmista dataa käytettäväksi. Staattisen verkkosivun tarjoamiseen valitsin S3.
Seuraavaksi yhdistin S3 ja DynamoDB lisäämällä niiden väliin API Gatewayn ja Lambda-funktion. Viimeisenä integroin WAF:n API:n suojaukseen ja mahdollisuuksien mukaan olisin integroinut myös CloudFrontin.

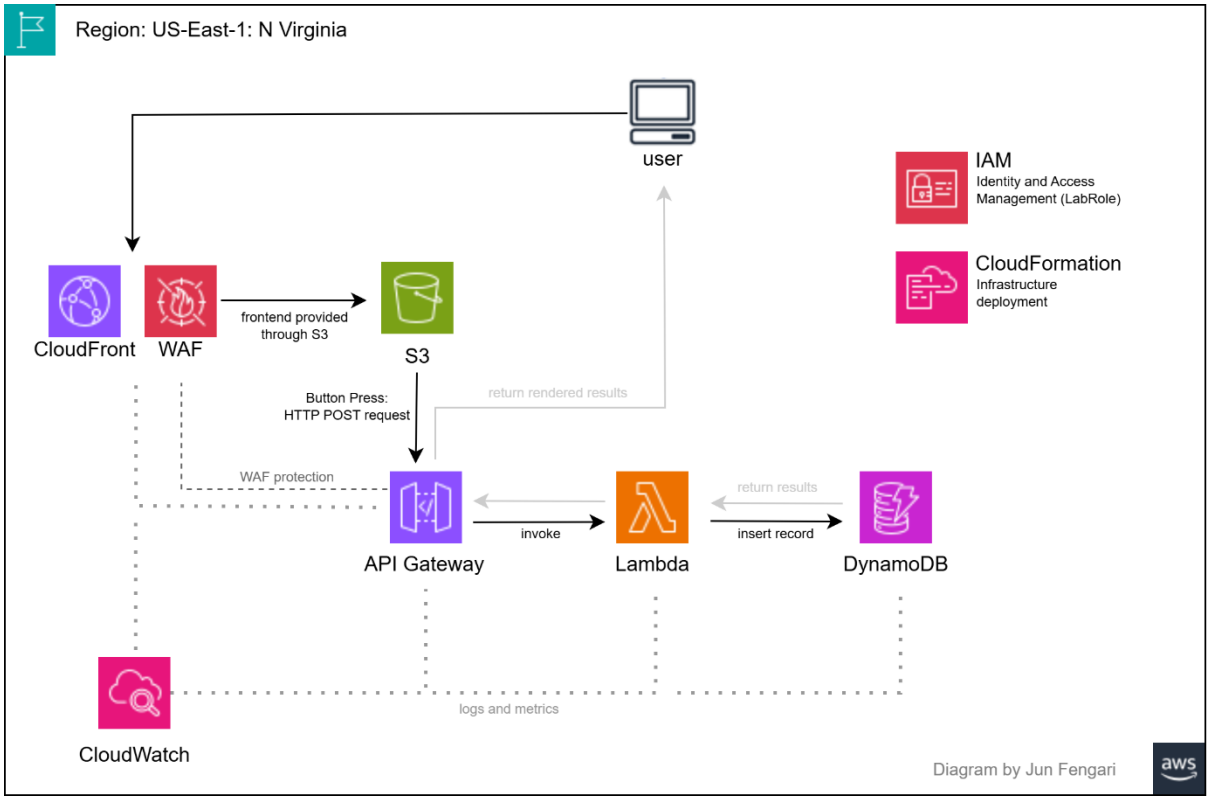
Lukulähteinä serverless palveluiden tutustumiseen käytin:

- (*Understanding serverless architectures - Optimizing Enterprise Economics with Serverless Architectures*, ei pvm.)
- (*Serverless Architecture*, 2026)

Mitä opit kurssin aikana, jotka auttoivat sinua pilviarkkitehtuurisi suunnittelussa ja toteutuksessa?

Kurssin aikana keskityttiin pääosin perinteisiin AWS ratkaisuihin, mutta moduulissa 14 käytiin serverless arkkitehtuuria läpi. Tämä moduuli ja liittyvät labrat auttoivat hahmoittamaan lopputyön peruskonsepteja.
Lopputyön toteutuksessa opin ja hahmoittelin myös monta uutta asiaa, erityisesti käytettyjen palveluiden toimintaa tarkemmalla tasolla.

Arkkitehtuurikaavio



Kuinka käytit draw.io-ohjelmaa arkkitehtuurikaavion luomiseen?
Mitä hyötyjä koit sen käytöstä?

Käytin draw.ion perustoimintoja sekä AWS-ikoneita (*Architecture Icon Page*, ei pvm.) kaavion toteuttamiseen.
Draw.io oli tosi helppokäyttöinen ja hyödyllinen arkkitehtuurien suunnitteluun ja hahmoittamiseen visuaalisesti, mikä syvensi ymmärrystäni valitsemastani aiheesta.

CloudFormation

CloudFormation template tiedostot löytyy *CloudFormation_Templates* kansiosta. Arkkitehtuurissa käytetyt template-tiedostot ovat näkyvillä suoraan kansion alla, ja alikansiossa *future-implementations* löytyy template pohjia joita ei käytetty.

Perustele minkälaisiin loogisiin kokonaisuuksiin/rakenteisiin muodostit CloudFormation-templaatit, ja miksi?

Erittelin tiedostot *Frontend*, *Backend*, *Database*, ja *Security* kokonaisuuksiin. Tämä on selkeä jako sovelluksen eri osista - eli käyttöliittymän, logiikan, tietokannan ja turvallisuuden kokonaisuudet.

Frontend: frontend.yaml

Tämä template ajetaan kolmantena.
Resurssit:

- **S3 bucket** → Sisältää ja tarjoaa staattisen nettisivun (HTML + JavaScript)

Backend: backend.yaml

Tämä template ajetaan toisena, sillä se riippuu tietokannan olemassaolosta.
Resurssit:

- **API Gateway** → REST API:n luonti, joka yhdistää käyttäjän pyynnön Lambda-funktioon.
- **Lambda function** → Tietokantaan kirjoittava Lambda-funktion luonti ja looginen sisältö.

Database: database.yaml

Tämä template ajetaan ensimmäisenä.
Resurssit:

- **DynamoDB table** → Tietokannan ja KasviHuone taulun luonti.

Security: waf.yaml

Tämä template ajetaan viimeisenä.
Resurssit:

- **WAF** → Suojaa API-Gatewaytä rajaamalla liikennettä.

Mitä etuja näit tämän lähestymistavan käytöstä?

Templaatit mahdollistivat ympäristön nopeata pystyttämistä – eli trail-and-error prosessilla pystyi nopeasti kokeilemaan, testaamaan ja poistamaan erilaisia ratkaisuja. Kaikki yhteenliittyvät palvelut ja prosessit ovat hahmoitettavissa yhdessä, mikä helpottaa ymmärtämistä ja virheiden korjaamista. Konsolissa sama toiminta olisi vaatinut paljon enemmän aikaa ja osaamista.

Prosessi

Alla kuvailen prosessia jolla sain arkkitehtuurini toimimaan toivotusti. Prosessin läpikäydessä avaan tarkemmin seikkoja liittyen asennukseen ja esimerkiksi template-tiedostojen sisältöä. Tätä prosessikuvausta voidaan käyttää myös pystytys- ja testausohjeena.
Tutustu github repositorioon:
https://github.com/JunFengari/AWS_Serverless_Architecture
Kloonaa tarvittaessa repositorion koneellesi.

1. Avaa konsolissa CloudFormation, ja luo uusi CloudFormation template ***database.yaml*** tiedoston pohjalta. Tämä pystyttää DynamoDB taulun ”KasviHuone”.

2. Tietokannassa ei ole vielä dataa, niin luodaan sille esimerkkidataa seuravalla CloudShell komennolla:

```
aws dynamodb batch-write-item --request-items '{
 "Kasvihuone": [
  {
   "PutRequest": {
    "Item": {
     "KasvihuoneID": {"S": "GH1"},
     "AikaLeima": {"S": "2026-03-14T12:00"},
     "LaiteID": {"S": "LAMPO01"},
     "LaiteTyyppi": {"S": "lampotila"},
     "Mittausarvo": {"N": "22.5"}
    }
   }
  },
  {
   "PutRequest": {
    "Item": {
     "KasvihuoneID": {"S": "GH1"},
     "AikaLeima": {"S": "2026-03-14T12:05"},
     "LaiteID": {"S": "KOST01"},
     "LaiteTyyppi": {"S": "kosteus"},
     "Mittausarvo": {"N": "65"}
    }
   }
  }
 ]
}'
```

Voidaan tarkistella dataa:

Table: Kasvihuone - Items returned (2)

Scan started on April 02, 2026, 14:10:47

Actions ▼ | Create item

KasvihuoneID *(String)*	AikaLeima *(String)*	LaiteID	LaiteTyyppi	Mittausarvo
GH1	2026-03-14T12:00	LAMPO01	lampotila	22.5
GH1	2026-03-14T12:05	KOST01	kosteus	65

3. Seuraavaksi tehdään uusi CloudFormation template ***lambdaAPI.yaml*** tiedoston pohjalta. Tämä luo Lambda funktion ”WriteKasvihuoneLambda”, joka kirjoittaa dataa DynamoDB:n Kasvihuone tauluun.
   Tiedosto myös luo API-Gateway:n ”KasvihuoneAPI” jotta nettisivu ja Lambda funktio pystyvät keskenään kommunikoimaan.

4. Kun Lambda funktio ja API ovat luotuja, voidaan testata sen toimintaa ensin CloudShellin kautta seuraavalla komennolla:

```
curl -X POST \
 https://5qtbtl4fqb.execute-api.us-east-1.amazonaws.com/prod/write \
 -H "Content-Type: application/json" \
 -d '{
     "KasvihuoneID": "GH2",
     "AikaLeima": "2026-04-02T12:00",
     "LaiteID": "LAMPO99",
     "LaiteTyyppi": "lampotila",
     "Mittausarvo": 23.5
   }'
```

```
~ $ curl -X POST \
>   https://5qtbtl4fqb.execute-api.us-east-1.amazonaws.com/prod/write \
>   -H "Content-Type: application/json" \
>   -d '{
>       "KasvihuoneID": "GH2",
>       "AikaLeima": "2026-04-02T12:00",
>       "LaiteID": "LAMPO99",
>       "LaiteTyyppi": "lampotila",
>       "Mittausarvo": 23.5
>     }'
{"message":"Item written successfully","item":{"KasvihuoneID":{"S":"GH2"},"AikaLeima":{"S":"2026-04-02T12:00"},"LaiteID":{"S":"LAMPO99"},"LaiteTyyppi":{"S":"lampotila"},"Mittausarvo":{"N":"23.5"}}}~ $
```

Tarkistellaan tuloksia:

Table: Kasvihuone - Items returned (4)

Scan started on April 03, 2026, 12:10:03

Actions ▼ | Create item

KasvihuoneID *(String)*	AikaLeima *(String)*	LaiteID	LaiteTyyppi	Mittausarvo
GH2	2026-04-02T12:00	LAMPO99	lampotila	23.5

5. Seuraavaksi voidaan luoda S3 bucket ***frontend.yaml*** tiedostolla, jälleen CloudFront template käyttäen. Huomio, jos testaat, joudut vaihtamaan S3 bucketin nimeä uniikiksi.

6. LabRole:n rajoituksista johtuen, muutetaan konsolin kautta luomamme S3 bucketin julkisuus asetuksia, ja lisätään bucket policy.
   Permissions välilehdessä poistetaan block public access täpän:

   Block public access (bucket settings)

   Public access is granted to buckets and objects through access control
   turn on Block all public access. These settings apply only to this bucke
   your applications will work correctly without public access. If you requ
   storage use cases. Learn more

   Block *all* public access
   ⚠ Off

   Sitten lisätään seuraava bucket policy, joka antaa browserille oikeuden hakea ja näyttää index.html sisältöä nettisivun vierailijalle:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::kasvihuone-frontend-demo/*"
    }
  ]
}
```

7. *Website* hakemistossa löytyy ***index.html*** tiedosto, jossa on yksinkertaisuuden vuoksi sekä nettisivun html ja javascript koodi yhdistettynä. Javascript osuudessa on API Endpoint, sekä nappi-logiikka.
   Lataa tämä tiedosto ”kasvihuone-frontend-demo” S3 buckettin.

8. S3 bucketin Properties välilehdestä muutetaan Static Website Hosting asetus -> Enabled.

9. Static Website Hosting osiosta myös löytyy sivuston linkki:
   http://kasvihuone-frontend-demo.s3-website-us-east-1.amazonaws.com/
   Tämän sivuston kautta pystytään testaamaan ”Write Test Record” nappia, joka kirjoittaa DynamoDB Kasvihuone tauluun. Jos kaikki onnistu, napin painallus palauttaa ”Success! Record written: ...”
   Tulos:

   Kasvihuone Write Demo

   Welcome to Jun's test website for Cloud Architecture!

   Press the button to write a test record to DynamoDB.

   Write Test Record

   Success! Record written: {"message":"Item written successfully","item":{

10. DynamoDB:stä voidaan käydä tarkistamassa kirjoitusta:

Table: Kasvihuone - Items returned (5)

Scan started on April 03, 2026, 13:11:32

Actions | Create item

KasvihuoneID *(String)*	AikaLeima *(String)*	LaiteID	LaiteTyyppi	Mittausarvo
GH_TEST	2026-04-03T10:11:24....	TEST01	lampotila	99
GH2	2026-04-02T12:00	LAMPO99	lampotila	23.5
GH1	2026-03-14T12:00	LAMPO01	lampotila	22.5
GH1	2026-03-14T12:05	KOST01	kosteus	65

11. Nettisivussamme ei ole tällä hetkellä suojausta, joten ideaalitilanteessa lisättäis S3 ja API:n suojauksena CloudFront ja WAF eteen, kuten arkkitehtuurikuvassa näkyy. Valitettavasti CloudFront ei ole käytettävissä LabRolen rajoituksista johtuen. Jos se olisi käytettävissä, *CloudFormation_Templates* kansion sisällä on *future-implementations* kansio, jossa löytyy valmis (testaamaton) ***security.yaml.***

12. Sen sijaan, voidaan luoda WAF:ia käyttäen suojausta meidän API:lle. *CloudFormation_Templates* kansiossa on ***waf.yaml*** tiedosto, jonka pohjalta luodaan viimeisen CloudFormation template. Sillä luodaan RateLimitRule, joka määrittää kuinka monta pyyntöä käyttäjä voi lähettää API:lle, ennen kuin WAF estää IP:osoitteen. Tämä suojaa DDOS hyökkäyksiltä, ja suojaa meidän lompakkoa, sillä napin painallukset aktivoi AWS:ssä maksullisia toimintoja (Lamdba funktion invokaatio, API pyynnöt, DynamoDB kirjoituksia).

    Tämän lisäksi, voidaan asettaa suoraan APIn asetuksiin rajoituksia API kutsuihin kuten näkyy kuvasta:

Throttling settings

Throttling
Limit the rate that users can call your API.

Rate
Enter the rate, in requests per second, that clients can call your API.
1
requests per second

Burst
Enter the number of concurrent requests that clients can make to your API.
1
requests

Tulokset & Reflektio

Video esittely: https://youtu.be/5Yj0DStf_X0

Arkkitehtuuriin kuuluvat tiedostot ovat saatavilla githubista: https://github.com/JunFengari/AWS_Serverless_Architecture

Sivuston väliaikainen linkki: http://kasvihuone-frontend-demo.s3-website-us-east-1.amazonaws.com/

Kuinka monimutkainen arkkitehtuurisi oli?

Arkkitehtuuri on mielestäni sopivan yksinkertainen, ja sisältää kaikki tarvittavat palvelut toimivaan serverless-ratkaisuun.
Toteutus oli itselle kuitenkin haastava, erityisesti koska se oli suurimmaksi osaksi koodipainotteista.

Mitä haasteita kohtasit suunnitellessasi ja toteuttaessasi sitä?

Haasteellisin tehtävä oli suunnittelu. Alussa tuntui vaikealta hahmoittaa mistä lähteä liikkeelle, ja miten palvelut toimii yhdessä käytännössä ja teknisellä tasolla.

Mitä tekisit toisin, jos aloittaisit alusta?

Jos tekisin uudelleen alusta, kokeilisin rohkeammin palveluita ja erilaisia toteutuksia konsolissa suoraan. Suunitteluvaiheessa olin tosi epävarma ratkaisuista ja koodi oli tosi vaikeata ymmärtää. Näitä huolia olisi voinut vähentää kokeilemalla kaikkea alusta asti käytännössä.

Noudattaako arkkitehtuuri Well-Architected Frameworkin periaatteita? Miltä osin kyllä, miltä osin ei?

Serverless arkkitehtuurin valinta tukee Well-Architected periaatteiden noudattamista: käyttämäni palvelut ovat AWS:n-hallinnoimia, skaalautuvat automaattisesti, ja tarjoavat korkean käytettävyyden (Operational Excellence & Reliability).
Serverless arkkitehtuurini toteuttaa myös Performance Efficiency, Cost Optimization ja Sustainability pilareita, koska palvelut suoritetaan vain tarvittaessa ja niistä veloitetaan käytön mukaan.
Suurin puute on mielestäni Security-pilari, vaikka implementoin perussuojausta (WAF, throttling), tuotantoympäristössä tähän kiinitettäisiin paljon enemmän huomiota.

Kuinka korkean käytettävyyden ja luotettavuuden periaatteita hyödynnetään ympäristössäsi?

S3 ja DynamoDB replikoivat sisällön automaattisesti useisiin AZ:hin, ja pyynnöt/failover tilanteet ohjataan terveisiin palvelimiin.
API-Gateway skaalautuu automaattisesti ja toimii useassa AZ:ssa. Se tukee myös automaattista failoveria ja mahdollistaa throttlingia kuorman hallitsemiseksi. Lambda-funktiot toimivat myös useassa AZ:ssa.

Suurimpana rajoituksena on, että koko sovelluksen toiminta riippuu yhdestä AWS-regionasta. Sallivammilla IAM-rooleilla voitaisiin toteuttaa multi-region replikoinnin S3 bucketille sekä DynamoDB:lle, mikä parantaisi käytettävyyttä ja vikasietoisuutta.

Lähteet & Tekoälyn Käyttö

Architecture Icon Page. (ei pvm.). Amazon Web Services, Inc. Noudettu 3. huhtikuuta 2026, osoitteesta https://aws.amazon.com/architecture/icons/

Serverless Architecture. (2026, maaliskuuta 12). GeeksforGeeks. https://www.geeksforgeeks.org/system-design/serverless-architectures/

Understanding serverless architectures—Optimizing Enterprise Economics with Serverless Architectures. (ei pvm.). Noudettu 3. huhtikuuta 2026, osoitteesta https://docs.aws.amazon.com/whitepapers/latest/optimizing-enterprise-economics-with-serverless/understanding-serverless-architectures.html

Tekoälyn käyttö

Käytin projektin suunnittelun aikana tekoälyä konseptien ja teknisen toimivuuden hahmoittamiseen ja ymmärtämiseen, sekä ideoiden sparraamiseen.

Toteutusvaiheessa generoin projektissa käytetyn koodin tekoälyllä (mukaan lukien CloudFormation templaatit, CloudShell komennot, osittain HTML ja JavaScript, sekä Lambda-funktio). Koodiin on jälkeenpäin tehty muutoksia omatoimisesti.
Tekoälystä oli myös apua vikatilanteiden ymmärtämiseen ja korjaamiseen.

Viimestelyvaiheessa tarkistin tekoälyllä kirjoitusvirheet ja kieliasun.